

6.7.2 Kragten Iterative Algorithm

Were Eq. (6.18) to be applied, it would be necessary to calculate the N partial derivatives related to each input quantity. If the function which links the output and input variables is complex, this becomes very problematic, if not impossible. All functions do not have partial derivatives. GUM proposes different numerical methods to perform this operation without being obliged to go through the partial derivatives in an explicit form. For example, the Monte Carlo method and the Bootstrap algorithm are accepted alternatives [9].

The following proposed iterative algorithm is even simpler. The solution is based on discretization of the partial derivatives and can be developed in a worksheet. We will call it the Kragten's algorithm since it was first published by that statistician, member of the GUM committee [10]. In the original paper, the algorithm was described and applied to quite a simple model, i.e. the linear inverse prediction model, but it is adaptable to any measurement model.

However, if this is complex, it is better to use a somewhat more advanced programming language such as Python. Resource K is the application of the Kragten iterative algorithm to the complex LEAD measurement model described by Eqs. (6.8) and (6.9). Like other Python programs here, measurement values are recorded in the code but they could be read from an external file.

The starting point of the algorithm is to calculate the initial value of the output quantity, denoted Z_0 using the measurement values of all input quantities. An iteration loop going through the N input quantities is used to calculate N modified values Z_n^* by adding to each input quantity G_n a slight specific variation $\Delta(G_n)$. To simplify things, the $\Delta(G_n)$ values are equal to one standard uncertainty $u(G_n)$ of each quantity. This means that all $u(G_n)$ are available as recommended at stage 3 of the GUM procedure.

At each step of the iteration loop, the new lead concentration is computed Z_n^* with the modified value of one G_n . The difference between the initial value Z_0 and the new value Z_n^* is calculated then squared. The sum of these N squared differences approximately corresponds to the standard variance (Eq. 6.22).

$$u_c^2(Z) \approx \sum_{n=1}^N (Z_0 - Z_n^*)^2 \quad (6.22)$$

This algorithm is applied to the LEAD example and illustrated by Resource K.

Resource K Iterative algorithm applied to LEAD (Python).

```
import pandas as pd
import numpy as np
```

The `Pb_Concentration` function defines the measuring model of the LEAD application and must be modified for any other measurement model. Each input quantity is named and each corresponding observed values is transferred by the

(Continued)

means of an array called `measure`. This is the simplest way to do so that each input quantity can be iteratively modified.

```
def Pb_Concentration(measure):
    Ms = measure[0]
    Cs = measure[1]
    Mp = measure[2]
    Wa = measure[3]
    Ar6 = measure[4]
    Ar8 = measure[5]
    Rr = measure[6]
    Rp = measure[7]
    As6 = measure[8]
    As8 = measure[9]
    Ap6 = measure[10]
    Ap8 = measure[11]
    K = (Ar8 / Ar6) / Rr*Rp
    return(Ms*Cs / (Mp*Wa) * ((K*As6 - As8) / (Ap8 - (K*Ap6))))
```

A set of measured input quantities is used to illustrate the algorithm and compute the initial value. It must be changed if another measurement of lead is achieved.

```
Gn = np.array([0.7806,0.41495,0.4944,0.9255,40.089,
40.0954,1.0189,0.8994,0.99979,0.00013,0.2454,0.52903])
Gu = np.array([0.0002,0.0034,0.0002,0.00288675,0.0036,
0.00385,0.0041,0.0036,0.0000125,0.00001,0.001,0.0021])
```

The initial lead concentration is calculated by using the initial values and displayed.

```
Z0 = Pb_Concentration(Gn)
print(Z0)
dev_sqrt =[]
```

The loop of iterations takes one by one input values stored in `Gn` and add one standard uncertainty stored in `Gu`.

```
for n in range(len(Gn)):
```

The initial value of the input quantity G_n is saved in a temporary variable to be restored after modification.

```
in_old = Gn[n]
```

One standard uncertainty is added to the measure.

```
In_new = Gn[n] + Gu[n]
Gn[n]=in_new
```

The modified concentration is calculated.

```
Z = Pb_Concentration(Gn)
```

The squared difference between initial and modified concentration is saved in an array.

```
Sqrt_dif = (Z - Z0)**2
dev_sqrt.append(sqrt_dif)
```

The initial value of the input quantity G_i is restored.

```
Gn[n] = in_old
```

The loop is closed, and the uncertainty budget assessed.

```
Budget=[]
total_squares = sum(dev_sqrt)
for i in range(len(dev_sqrt)):
    budget.append(100*dev_sqrt[i]/total_squares)
```

Table 6.2 can thus be completed by the Table 6.3. The column G_n contains the values used to calculate the initial value of Z_0 . The column $u(G_n)$ lists the standard uncertainties. The initial value is $Z_0 = 1.999982505$ mg/kg. The modified results are obtained by iteratively adding to each input quantity, taken one by one, one standard uncertainty $u(G_n)$ which gives $G_n + u(G_n)$.

Table 6.3 LEAD – Calculation of the combined standard uncertainty with the iterative algorithm.

Step		G_n	$u(G_n)$	Modified value Z_n^*	Squared difference	Contribution (%)
0				$Z_0 = 1.999982505$		
1	Ms	0.7806	0.0002	2.00869	2.63×10^{-7}	0
2	Cs	0.41495	0.0034	2.02464	2.71×10^{-4}	30
3	Mp	0.4944	0.0002	2.00737	6.59×10^{-7}	0
4	Wa	0.9255	0.00288675	2.00194	3.90×10^{-5}	4
5	Ar6	40.089	0.0036	2.00788	9.33×10^{-8}	0
6	Ar8	40.0954	0.00385	2.00851	1.07×10^{-7}	0
7	Rr	1.0189	0.0041	1.99459	1.85×10^{-4}	21
8	Rp	0.8994	0.0036	2.02183	1.86×10^{-4}	21
9	As6	0.99979	0.0000125	2.00821	6.31×10^{-10}	0
10	As8	0.00013	0.00001	2.00816	5.18×10^{-10}	0
11	Ap6	0.2454	0.001	2.01387	3.24×10^{-5}	4
12	Ap8	0.52903	0.0021	1.99477	1.80×10^{-4}	20
Combined standard variance: $u_c^2(Z)$					8.94×10^{-4}	100.00
Combined standard uncertainty: $u_c(Z)$					0.02993	

For abbreviations see Table 6.2.

Source: Adapted from Kragten [10].

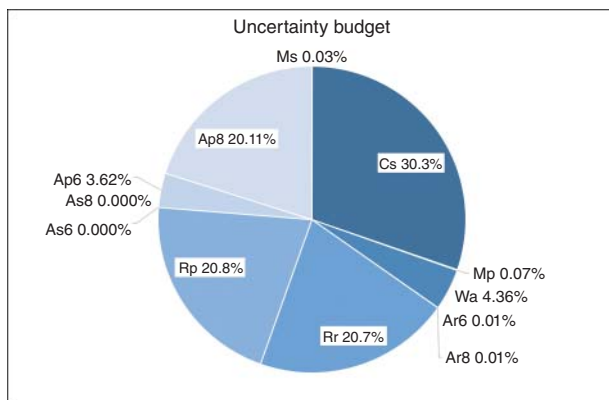


Figure 6.4 LEAD – uncertainty budget.

The penultimate column contains the squared differences $(Z_0 - Z_n^*)^2$. As these differences can be ridiculously small numerically speaking, for these calculations it is necessary to use double precision variables. From these data, it is also possible to calculate the number of degrees of freedom associated with this estimate and use it to compute the coverage factor. This extension of the algorithm is not presented but available in the original publication. It was preferred to apply the standard GUM coverage factor for this example.

Most importantly, it is now possible to establish the uncertainty budget, described in the GUM as an indispensable complement to the MU estimate. The relative contribution of each quantity to the total combined uncertainty – its relative sensitivity coefficient – is obtained by calculating the ratio of the squared difference to the total standard deviation, as illustrated in Figure 6.4 and reported in the last column of Table 6.3. For example, the contribution of Cs (lead concentration in the spike) to combined uncertainty is:

$$100 \times \frac{2.71 \times 10^{-4}}{8.94 \times 10^{-4}} \approx 30\%$$

It can be concluded that the spiked concentration accounts for about 30% of the total MU and that three other critical input quantities are the isotopic abundance ^{208}Pb in the test sample (*Ap8*), the ratios $^{208}\text{Pb}/^{206}\text{Pb}$ in the CRM and in the test sample (*Rr* and *Rp*). When possible, better control of these quantities could help to reduce the lead MU. The graphical representation, e.g. in sectors, allows to better highlight the quantities that are influential on the MU.

The example illustrates the MU estimation for a single lead measurement value. If another sample is analyzed or the measure repeated, the whole calculation must be repeated with the new values of G_n . This solution is uneasy for contract laboratories where highly variable samples and matrices are seen daily. To avoid this limitation, the proposed solution described in Section 7.5 is to establish an uncertainty function applicable to any new sample and directly estimate MU of any inverse-predicted concentration.

There are dozens of commercially available uncertainty calculators that allow the recurrent calculation of MU. Some are even under free license, but many are commercial. The tricky point remains the input of the measurement model which may require some programming knowledge.

6.8 Calculate Expanded Uncertainty

The expanded uncertainty $U(Z)$ is simply “the product of a combined standard MU and a factor larger than the number one.” This is then directly derived from standard uncertainty $u_c(Z)$ as indicated by formula (6.3) where k_{GUM} represents the GUM standardized coverage factor, which depends on a coverage probability. Expanded uncertainty is the most practical form of MU.

Combined uncertainty is not easily understood by end-users because it remains theoretical like a standard deviation. To make it more understandable and practical it is multiplied by a coverage factor and expressed as an interval. For instance, it is well known that ± 2 standard deviations include about 95% (precisely 97.7%) of the data if they are exactly distributed according to a Normal law. The coverage factor plays the same role and depends on the expected probability of the resulting coverage interval.

This is the most interesting expression of MU while a coverage interval is an “interval containing the set of true values of a measurand with a stated probability, based on the information available.” Therefore, coverage interval can directly be used to express the analytical result:

Expanded uncertainty

$$U(Z) = k_{GUM} \times u_c(Z) \quad (6.3)$$

Coverage interval

$$Z \pm U(Z) \quad (6.5)$$

When the number of degrees of freedom (df) is accurately known at the end of the estimation of the MU, it can be used to have an exact value for the coverage factor. In most examples presented here, it is the quantile of the Student's t law, for the given probability and the number of df . This point is developed more in Section 7.2.

When the information is not available, the standardized GUM coverage factor denoted k_{GUM} is an interesting and easy solution. It can take simple conventional values:

Value of k_{GUM}	Coverage probability	Contents of the coverage interval
$k_{GUM} = 2$	95%	95% possible true values of Z
$k_{GUM} = 3$	99%	99% possible true values of Z

For example, for the determination of lead by ICP-ID-MS, the test material contains $C_p = 1.99998$ mg/kg, and the combined standard uncertainty